

MILESTONE 4

SW Engineering CSC648-848 Spring 2025

Buy-Sell WWW app - GatorMarket

Team 2

Claudia Wormley (Team Lead)

Nathan Donat-Filliod (GitHub Lead)

Daniel Cervantes (Back-End Lead)

Davis Rosenstein (Front-End Lead)

Fatimah Abdolcader

cwormley@sfsu.edu

date of submission: 5-21

1) **Product summary**

Name: Gator Market

Product Description:

GatorMarket is a marketplace designed exclusively for the San Francisco State University community, allowing students, faculty, and staff to buy and sell items like textbooks, electronics, and furniture. What makes it unique is its exclusive access via SFSU email, a class-specific search for course materials, and safe on-campus pickup locations—all tailored to the campus experience. Created by SFSU computer science students, GatorMarket emphasizes simplicity, security, and community connection, standing apart from platforms like Craigslist or Facebook by focusing only on what SFSU users need.

Finalized P1:

Unregistered users

1. Unregistered Users shall be able to browse all available listings without an account.
2. Unregistered Users shall be able to search for items by title, category, or keywords.
3. Unregistered Users shall be able to search items by category and text input.
4. Unregistered Users shall be able to view item descriptions, images and item status (available, pending, sold).
5. Unregistered Users shall be able to register.
6. Unregistered Users shall be able to search for books based on specific SFSU courses (e.g., CSC 648-848) to find textbooks for their classes.

Registered users

1. Users have all of the abilities of Unregistered Users
2. Users shall be able to specify a meet up place (e.g., library, student center) for easy trading.
3. Only members of the SFSU community shall be allowed to sign up and log in using their university email.
4. Users shall be able to create a new listing with a title, description, price, category, and images and post it (subject to admin approval).
5. Users shall be able to message the seller.
6. Sellers shall have a dashboard showing all their active listings and messages.

7. Sellers shall be able to delete an item once a transaction is complete.
8. Users shall be able to login.
9. Registered Users shall be able to remove their active listings from their dashboard.

Admin

1. Admin shall be required to review and approve an item before becoming public.
2. Admin shall be able to delete users.
3. Admins shall have the ability to delete listings that violate platform policies

Website Url: <http://44.201.159.31/>

2) Usability test plan for selected function

Test usability of search function

a. Test objectives:

The objective of this usability test is to evaluate the effectiveness, efficiency, and satisfaction of users when using the search function. We aim to identify any difficulties users face, assess the clarity of the interface, and determine whether the search results meet user expectations.

b. Test background and setup:

The platform is a web-based marketplace where students at SFSU can list and search for items such as textbooks, furniture, and electronics. The search bar appears at the top of each page. The system will be accessed by a web browser (Chrome or Firefox). Testers will begin on the platform's homepage without being logged in. All tests should be completed using only the search function. Testers will need access to a laptop or desktop with an internet connection as well as a keyboard and mouse or touchpad.

Intended users are college students at SFSU who wish to buy or sell with their fellow students on campus. Testers will include first time and returning users of the website.

The URL of the system is <http://44.201.159.31>

The test environment can occur in either a home or lab setting, depending on tester availability. No live monitoring or cameras need to be used during the test.

Test sessions will not require training prior to the test. A task description will be provided followed by a Likert scale feedback questionnaire.

c. Usability Task description

Instructions for Usability Tester:

You are going to test the search function for GatorMarket, a buy and sell website for college students. Please follow each task in order.

Task 1:

- Start from the homepage
- Search for a CSC book
- Message the seller of a listing

Task 2:

- Start from the homepage
- Change the category
- Search for item relevant to category
- Message the seller of a listing

Task 3:

- Start from the homepage
- Change the category
- Search for item outside of category
- See recent posts

d. Plan for evaluation of Effectiveness:

We will check how many of the search tasks users can complete without any help. If they get the right result after searching, that counts as a success. If they get stuck or click on the wrong thing, that will be something we will note.

e. Plan for Evaluation of efficiency:

We will time how long it takes users to complete each search task. If they finish quickly without clicking around too much or getting confused, that indicates the search feature is efficient. In addition to tracking completion time, we will also count the number of clicks and page navigations needed for each task. Fewer clicks and minimal navigation suggest a more efficient design. If users hesitate, take longer, or click excessively, we will investigate the reasons.

f. Plan for Evaluation of user satisfaction (Likert scale questionnaire):

After testing, we'll ask the user to rate their experience using the search bar. These are the questions they'll answer:

1. "It was easy to find and use the search bar."
2. "The search results were what I expected."
3. "I'd use this search function again if I needed to find something."

Each question uses a 5-point scale: Strongly Agree, Agree, Neutral, Disagree, Strongly Disagree

g. GenAI use:

We used ChatGPT (GPT-4) to help us write and organize this test plan. It helped us make sure our explanations were clear and matched class guidelines. For example, we asked it how to write a usability test plan and how to phrase Likert scale questions. We think it was really helpful. Utility Rank: HIGH

3) QA test plan and QA testing - about 2 pages

a. Test objectives:

The objective is to confirm that the Gator Market web application's Search feature operates as intended. The test will check whether the UI is responsive across browsers, whether partial matches are handled effectively, and whether search results accurately match the entered terms.

b. HW and SW setup

Hardware: MacBook Pro (M1), 16GB RAM

Browsers Used: Google Chrome (v124), Mozilla Firefox (v125)

Operating System: macOS Sonoma

Test URL: <http://44.201.159.31>

c. Feature to be tested

Search Function on the Gator Market homepage to find listed items (books, etc.)

d. QA Test plan and Results:

Test #	Test Title	Test Description	Test Input	Expected Output	Chrome Result	Firefox Result
--------	------------	------------------	------------	-----------------	---------------	----------------

1	Search Exact Title	Test full exact match of item name	"French Book"	Display 1 item titled "French Book"	PASS	PASS
2	Search Partial Match	Test partial input for item	"Fren"	Display 1 item matching "Fren"	PASS	PASS
3	Search No Match	Search for non-existent item	"Biology"	No results or message: "No items found"	PASS	PASS
4	Empty Search	Test search with no input	"" (empty)	Show all available items (default behavior)	PASS	PASS
5	Special Characters	Test search with invalid/special characters	"@@@@"	No results or error handling	PASS	PASS

e. GenAI use

Tool Used: ChatGPT (GPT-4-turbo, May 2024 version)

I used ChatGPT to generate a structured QA test plan for the "Search" functionality. It helped me define common edge cases, standardize test result formatting, and provide examples for expected outputs. This saved time and improved clarity for the report.

Example Prompt:

"Generate a QA test plan table with 5 test cases for a website search feature, using Chrome and Firefox."

Utility Ranking: HIGH – The GenAI tool helped quickly organize and format the document *while ensuring important scenarios were included*.

4) Peer Code Review

Code Review Communication:

Re: Team 02 Code Review

NP

© Nathan Pierre Jean Donat Filliod <ndo... Today at 1:37 PM

To: © Claudia Jane Wormley

ChatGPT - Answer.pdf
154.7 KB +1 more

Download All · Preview All

Hi Cj,

Thanks for the update and for completing the API call integration.

I've reviewed the implementation on [PR #51](#), and here are my main comments:

- It would be better to remove alerts because it slows the process. And replace it by in-page error/success messages.
- Daniel's description is missing in the about section.
- I added field **user_contact** in **Message** table to fit with your code.

I've also attached to this email both the proof of my review on GitHub and a detailed code review generated with ChatGPT for additional insights.

Let me know if you'd like to discuss any of my comments further.

Best,
Nathan
Team 02 – GitHub Master

Team 02 Code Review

CJ

© Claudia Jane Wor... Today at 11:52 PM

To: © Nathan Pierre Jean Donat Filliod

Hi Nathan,

I have completed API calls for database access in frontend. All implementation has been finished. Frontend team will review the frontend functionality.

Please look over the new implementation and send back your comments. Ensure that this is ready for main branch and for server implementation.

<https://github.com/CSC-648-SFSU/csc648-sp25-03-team02/pull/51>

Thank you!
Cj, Team 02

Review Comments:

 **DonatNathan** approved these changes [13 minutes ago](#) [View reviewed changes](#)

DonatNathan left a comment ...

Great work!
The only useful changes would be to remove the alerts and replace them with in-page messages.



```
application/src/frontend/src/messageseller.js
```

```
62 +         }},
63 +     });
64 +     const result = await response.json();
65 +     alert('Message sent to seller successfully!');
```

 **DonatNathan** [29 minutes ago](#) ...

Maybe it's a better idea to remove the alert or replace it by a message in-page.
It looks slower with an alert.



```
application/src/frontend/src/postitem.js
```

```
58 +     }
59 +
60 +     const result = await response.json();
61 +     alert('Post created successfully! \n Please give our team 24 hours to review before');
```

 **DonatNathan** [29 minutes ago](#) ...

Replace the alert by a confirm message in-page.



 Reply...

Resolve conversation

```
application/src/frontend/src/signandreg/register.js
```

```
80 +     })
81 +     .catch((err) => {
82 +         console.error('Error registering user:', err);
83 +         alert('An error occurred during registration. Please try again.');
```

 **DonatNathan** [27 minutes ago](#) ...

Same, it's better to return an error message under the form.



```
application/src/frontend/src/signandreg/register.js
75 +     navigate('/signin');
76 +   } else {
77 +     // Handle error messages from the backend
78 +     alert(data.message || 'Registration failed');
```

 **DonatNathan** 27 minutes ago ...

Same thing but with an error message.



 Reply...

Resolve conversation

```
application/src/frontend/src/signandreg/register.js
70 +     .then((data) => {
71 +       console.log('Registration response:', data);
72 +       if (data.message === 'Registration successful') {
73 +         alert(data.message);
```

 **DonatNathan** 26 minutes ago ...

I think it's better to remove the alert.
And maybe add an auto login when a user register.



 Reply...

```
application/src/frontend/src/signandreg/signin.js
19 +     try {
20 +       const data = await login(email, password);
21 +       console.log('Login successful:', data);
22 +       alert('Login Successful!');
```

 **DonatNathan** 26 minutes ago ...

Same as in the register.



 Reply...

Resolve conversation

```
application/src/frontend/src/signandreg/signin.js
24 +       window.location.reload();
25 +     } catch (error) {
26 +       console.error('Login failed:', error);
27 +       alert('Invalid credentials. Please try again.');
```

 **DonatNathan** 25 minutes ago ...

Same as in the register.



 Reply...

GenAI interaction:

PROMPT:

Can you review this React component for code quality, structure, readability, and best practices? Also, suggest improvements for performance, accessibility, and maintainability if needed.

✔ What You're Doing Well

◆ Good Practices

- ✔ **Separation of concerns:** Routes are modularized (`/routes/...`).
- ✔ **Environment variables** for secrets (`SESSION_SECRET`, `FRONTEND_URL`, etc.).
- ✔ **Middleware usage:** `express.json()`, `cors()`, `express-session`.
- ✔ **Health checks:** DB test routes are helpful for debugging.
- ✔ **SPA Support:** The `catch-all` route serves the React app correctly.

✔ Summary of Recommendations

Category	Suggestion
🔒 Security	Use <code>secure</code> and <code>sameSite</code> cookies properly for production
🔧 Clean Code	Group and re-export routes, reduce clutter in <code>server.js</code>
⚙️ Performance	Add <code>compression</code> middleware
⚠️ Errors	Add centralized error-handling middleware
🔍 Maintainability	Use <code>.env</code> for static paths and improve route folder structure
📊 Logging	Use <code>morgan</code> or <code>winston</code> for better request logging

GenAI Review:

I used the ChatGPT (GPT-4) Freemium Version for a quick Code Review of the project before deploying it on production. And, for this prompt, the answer was detailed and helped us to understand new good practices. That's why the utility of GenAI here is HIGH.

5) Self-check on best practices for security

Asset to be protected	Types of possible/expected attacks	Consequence of security breach	Your to mitigate/protect the asset
API Keys & Secrets	Unauthorized external access	Key leakage	Stored in environment variables, never committed to codebase.
Products Pictures	Malware injection	Unauthorized access, Storage overflow	Images are stored on the server . File paths are sanitized, and uploads are restricted to safe formats.
User Credentials	Brute-force attacks	Password theft, Credential stuffing	Passwords are hashed using crypto before storage. We use rate limiting and enforce strong passwords. And verification of e-mail including "sfsu.edu".
Application Source Code	Reverse engineering	Code leakage, IP theft	Use private Git repositories, access control via GitHub teams.
User Input Data	XSS, SQL injection	Application crashes	All input is validated and sanitized using server-side and client-side

			validation. And the search bar can accept input of up to 40 alphanumeric characters.
Terms	-	-	Users must explicitly accept terms and conditions.

6) Self-check of the adherence to original Non-functional specs

1. Application shall be developed, tested and deployed using tools and cloud servers approved by Class CTO and as agreed in M0 **DONE**
2. Application shall be optimized for standard desktop/laptop browsers e.g. must render correctly on the two latest versions of two major browsers **DONE**
3. All or selected application functions shall render well on mobile devices (no native app to be developed) **DONE**
4. Posting of sales information and messaging to sellers shall be limited only to SFSU students **DONE**
5. Critical data shall be stored in the database on the team's deployment server. **DONE**
6. No more than 50 concurrent users shall be accessing the application at any time **DONE**
7. Privacy of users shall be protected **DONE**
8. The language used shall be English (no localization needed) **DONE**
9. Application shall be very easy to use and intuitive **DONE**
10. Application shall follow established architecture patterns **DONE**
11. Application code and its repository shall be easy to inspect and maintain **DONE**
12. Google analytics shall be used **DONE**
13. No email clients or chat services shall be allowed. Interested users can only message to sellers via in-site messaging. One round of messaging (from user to seller) is enough for this application. **DONE**
14. Pay functionality, if any (e.g. paying for goods and services) shall not be implemented nor simulated in UI. **DONE**
15. Site security: basic best practices shall be applied (as covered in the class) for main data items **DONE**
16. Media formats shall be standard as used in the market today **DONE**
17. Modern SE processes and tools shall be used as specified in the class, including collaborative and continuous SW development and GenAI tools **DONE**

18. The application UI (WWW and mobile) shall prominently display the following exact text on all pages "SFSU Software Engineering Project CSC 648-848, Spring 2025. For Demonstration Only" at the top of the WWW page Nav bar. (Important so as to not confuse this with a real application). You have to use this exact text without any editing.
DONE

7) **Use of genAI tools like ChatGPT and copilot -Cj**

ChatGPT was used for this write up.

The genAI assisted in assuring that all parts of this assignment were completed in full. Given the detailed instructions and current document I was able to assess any changes that had to be made to complete all requirements. This was helpful but I still looked over the contents to make sure the document was up to team standards. Overall ChatGPT was helpful in offering suggestions and pointing out errors.

Given the following directions, review our current write up noting anything that needs to be improved.

Helpfulness: HIGH

ChatGPT was also used to make sure all nonfunctional requirements were met and helped in fulfilling ones that were incomplete. As some of the nonfunctional requirements were given I was able to select individual nonfunctional requirements for ChatGPT to see if it was being done properly. This included checking for site security.

Given this nonfunctional requirement, how can I ensure my project meets expectations?

I need to make this page render for mobile apps. This element is not rendering correctly. Given the element, fix the css code to make it reactive.

Helpfulness: HIGH